

Summary of weekly activities**Objectives:**

1. Proceed with the experimental/simulation work and data collection for the hardware-software integration phase.
2. Able to design and conduct the experiment/simulation to validate AHB-Lite to Quad-SPI protocol translation.
3. Choose the appropriate technique to achieve the simulation goal with proper justification regarding memory mapping and compiler behavior.

Activities & Summary**Simulation Design & Setup**

To transition the system from isolated Verilog module testing to full System-on-Chip (SoC) integration, I designed a simulation environment targeting the bare-metal C software driver executing on the simulated ARM Cortex-M3 processor. The experiment required compiling `main.c`, `qspi_driver.c`, and `startup.c` into a .hex format to be loaded directly into the ARM processor's ROM. I configured the VCS testbench to issue initial AHB-to-SPI memory write transactions, pointing the CPU strictly to the QSPI controller's base address space (0x40000000). This setup was established as a baseline to observe the resulting AHB bus signals (HADDR, HWDATA, HWRITE) and their corresponding physical SPI pin toggling on the output.

Technique Justification & Implementation

During initial implementations, standard C variable assignments were utilized to pass data to the controller. However, the ARM GCC compiler automatically mapped these local pointer variables to the processor's default stack memory space at 0x20010000. Because the current SoC testbench strictly isolates the flash controller and lacks an instantiated SRAM block, these writes fell into a void. To correct this without adding unnecessary IP blocks to the simulation, I chose to implement direct macro casting using volatile absolute memory pointers (e.g., `*((volatile uint32_t *) (FLASH_BASE + 0x0100)) = 0xDEADBEEF;`). This technique is highly justified because it completely bypasses the stack memory, forcing the CPU to hold the addresses exclusively within its internal registers (R0-R15) and routing the data directly to the AHB bridge.

Experimental Execution & Data Collection

I executed the VCS simulation and collected comprehensive waveform data for the SoC's internal busses and external interfaces. The experimental results successfully demonstrated a zero-software-overhead memory write. The data collection verified that the custom Verilog hardware autonomously intercepted the AHB write request and instantly stalled the CPU via HREADY. Furthermore, it verified that the Finite State Machine (FSM) generated the Write Enable (0x06) and Quad Page Program (0x34) commands autonomously without software intervention, successfully blasting the 32-bit 0xDEADBEEF payload across all four data lanes to the external flash memory model.

Student's Name/Signature: Date:	Supervisor's Name/Signature: Date:
--	---

Summary of weekly activities**Challenges & Solutions Challenge:**

The ARM processor initially triggered a HardFault exception during the write tests, falling into an infinite emergency crash loop (b . / 0xe7fe). Furthermore, the external flash memory was rejecting transactions because the controller was misinterpreting the C driver commands, blasting payload data as command bytes (e.g., attempting to write the 0x9F JEDEC ID directly into the flash array).

Solution: I traced the root cause of the bus lockup to the aforementioned stack memory void. The secondary issue regarding the command generation led to a fundamental discovery about the RTL architecture: the custom Verilog design acts as a highly optimized "Pure XIP" (Execute-In-Place) hardware accelerator bridge, rather than a standard MMIO peripheral with software-accessible FIFO registers. To solve the issue, I deleted the redundant C software driver entirely. By restructuring the software to rely purely on absolute memory pointers, the software aligned seamlessly with the hardwired XIP architecture, eliminating the HardFault crashes and allowing the hardware to natively generate the correct protocol.

Student's Name/Signature: Date:	Supervisor's Name/Signature: Date:
--	---